



Unidad 3

Herencia

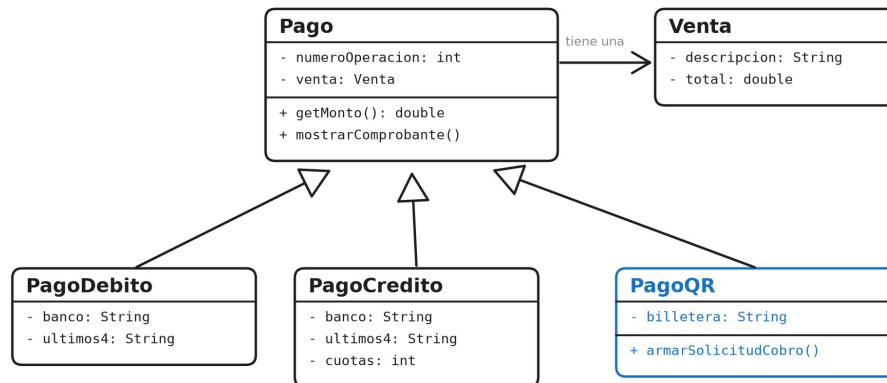
Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026



La pre clase, en dos preguntas

- PagoDebito y PagoCredito eran casi el mismo archivo. Y llegó el pedido de aceptar pagos con QR.
- ¿Qué costaba cada cambio mientras eran copias? ¿Quién avisaba si una regla se corregía en una clase y en la otra no?
- ¿Qué tuvo que escribir PagoQR para entrar al sistema? ¿Qué no tuvo que escribir?

La jerarquia de la pre clase



La flecha con triangulo es extends. La flecha simple es una referencia, composicion. PagoQR es el que llegó nuevo: entro heredando.

Cada cambio valía triple y cada olvido era invisible. La herencia ayuda en eso.

Un pago con QR es un pago



Lo que heredó

El número de operación, la venta, las validaciones del constructor, `getMonto()` y el encabezado del comprobante.

Sin escribir una línea.

Lo que escribió

Decodificar el código escaneado, armar la solicitud para la API de la billetera y su línea del comprobante.

Solo lo que un QR tiene de propio.

La relación ES UN se debe cumplir.

Lo que acaban de hacer



El catálogo decía que el autor de la notebook era Asus.
Nadie escribió esa línea para `Electronico`, vino entera con la copia de `Libro`.
Después del refactor el error desapareció solo. El encabezado vive una sola vez en `Producto` y cada clase escribe únicamente sus líneas propias.

--- Catalogo de la tienda ---

Codigo: ELE-001

Nombre: Notebook 14 pulgadas

Precio: \$1750000.0

Precio final (IVA inc.): \$2117500.0

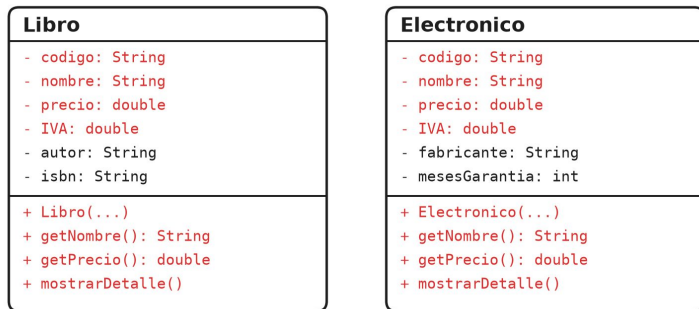
Autor: Asus

Garantia: 24 meses

El bug no era una línea mal escrita, era un método copiado entero.

El catálogo, antes y después

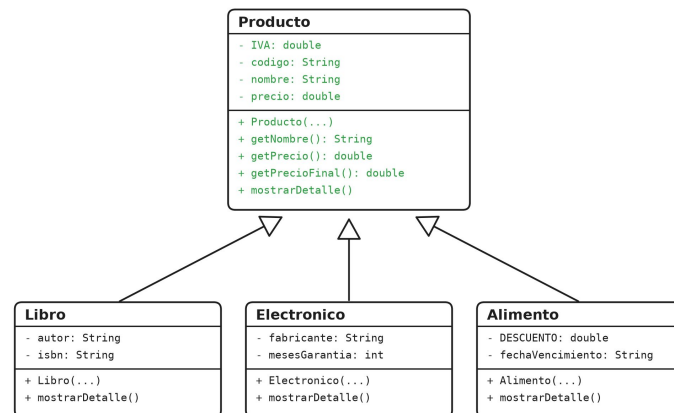
Antes: dos clases casi iguales



En rojo, lo que existe dos veces.
Y mostrarDetalle() ya divergio: "Autor: Asus".

Notacion UML: - private + public

Despues: lo comun vive una sola vez



La flecha con triangulo es extends. En verde, lo que los tres reciben sin escribir. Cada hijo declara solo lo suyo, y su `mostrarDetalle()` reutiliza el del padre con `super`.

La masa roja se convierte en un solo bloque verde. Y Alimento ni siquiera existía.

Qué se hereda con extends



Se hereda

Todos los atributos y los métodos. La subclase no copia el código del padre, lo tiene.

Un objeto `Libro` tiene adentro la parte `Producto` completa.

No se hereda

Los constructores. Cada clase escribe el suyo.
Los miembros `private` no se pueden nombrar desde el hijo, aunque están adentro del objeto.
Para eso quedan los getters.

Heredar no es copiar. Lo que es del padre sigue viviendo en el padre.

Para construir un hijo, primero el padre



- `super ( . . . )` llama al constructor de la superclase y tiene que ser la primera instrucción.
- Si el padre no tiene constructor sin argumentos, escribirlo es obligatorio. Sin `super ( . . . )` no compila.
- Ese paso garantiza que ningún producto se instancie sin pasar por las validaciones de `Producto`.

```
public class Alimento
    extends Producto {

    public Alimento(String codigo,
        String nombre, double precio,
        String vence) {

        super(codigo, nombre, precio);
        // primero el padre,
        // despues lo propio
    }
}
```

El orden es siempre el mismo, primero ejecuta el constructor del padre y después el del hijo.

protected, la tentación de la clase 3



La tentación

`Alimento` necesita el precio final para su oferta.
Pasar `precio` a `protected` resuelve rápido.
Y cualquier subclase, las de hoy y las futuras,
puede modificarlo sin pasar por las reglas del
padre.

La respuesta

El `getPrecioFinal()` heredado alcanzó.
El padre sigue siendo el único responsable de su
estado, igual que en la clase 3.

El estado queda `private` siempre. `protected` es la excepción que hay que poder justificar.

Sobrescribir sin perder al padre



`super.mostrarDetalle()`

Reutiliza el encabezado común tal como lo
escribe `Producto`, sin copiarlo.

Las líneas propias

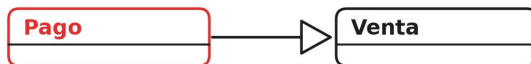
El autor y el ISBN en `Libro`, la garantía en
`Electronico`, el vencimiento y la oferta en
`Alimento`.

`@Override` no es obligatorio, pero avisa al compilar si la firma no coincide con la del padre.

Herencia o composición

- Herencia cuando la relación es “**es un**”. Un alimento es un producto.
- Composición cuando la relación es “**tiene un**”. Un pago tiene una venta, la referencia se guarda como atributo.
- `class Pago extends Venta` compila y da todos los datos de la venta. Pero esta mal.

~~x~~ `class Pago extends Venta`



compila, pero miente

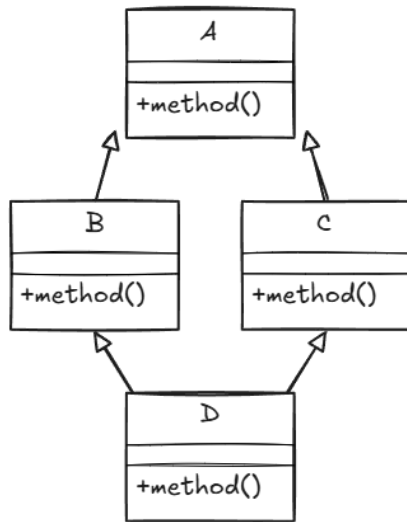
✓ `private final Venta venta`



el pago guarda la referencia
y colabora mandando mensajes

Una sola superclase

- En Java una clase hereda de una sola superclase. C++ o Python permiten varias, y aparece el problema del diamante.
- Si B y C sobrescriben el mismo método de A, ¿cuál de las dos versiones hereda D?
- Java lo prohíbe de raíz y ofrece otro camino para cumplir varios roles, las interfaces.



No hay respuesta buena para el diamante, por eso Java eligió no tener que responderla.

En resumen



- `extends` crea la jerarquía, `super ( . . . )` construye la parte heredada y la valida.
- Lo común se escribe una sola vez. Cada clase agrega solo lo suyo.
- El estado queda `private` y se lee con `getters`. `protected` solo con justificación.
- Herencia para “**es un**”, composición para “**tiene un**”.

Especializar sin copiar.

¿Existe producto?

`new Producto("PRO-001", ...)` compila y corre, y la tienda no vende productos genéricos. La herramienta para salvar ese problema en la próxima clase.
